

Bulk Operations using Z.EntityFramework.Extensions.EFCore Extension

In this article, I will discuss Bulk Operations in Entity Framework Core using Z.EntityFramework.Extensions.EFCore Extension with Examples. Please read our previous article, which discussed [Bulk Insert, Update, and Delete Operations in Entity Framework Core](#) with Examples.

Bulk Operations using Z.EntityFramework.Extensions.EFCore Extension

Efficiently handling large volumes of data in applications is crucial for performance and scalability. While Entity Framework Core (EF Core) provides robust features for data manipulation, performing bulk operations like insert, update, and delete can be challenging due to the lack of native support and multiple database round trips. The [Z.EntityFramework.Extensions.EFCore](#) library by ZZZ Projects bridges this gap by offering high-performance bulk operations, making it easier to manage large datasets efficiently.

What is Z.EntityFramework.Extensions.EFCore?

Z.EntityFramework.Extensions.EFCore is a third-party extension library developed by ZZZ Projects that enhances Entity Framework Core by enabling high-performance bulk operations, such as bulk inserts, updates, and deletes. Entity Framework Core does not natively support efficient bulk operations, which can lead to performance issues when working with large datasets. This extension helps overcome these limitations by reducing database round trips, optimizing commands, and improving performance for large-scale data modifications.

Bulk Operation Methods of Z.EntityFramework.Extensions.EFCore Package:

The following are the extended methods that we can use to perform Bulk Operations.

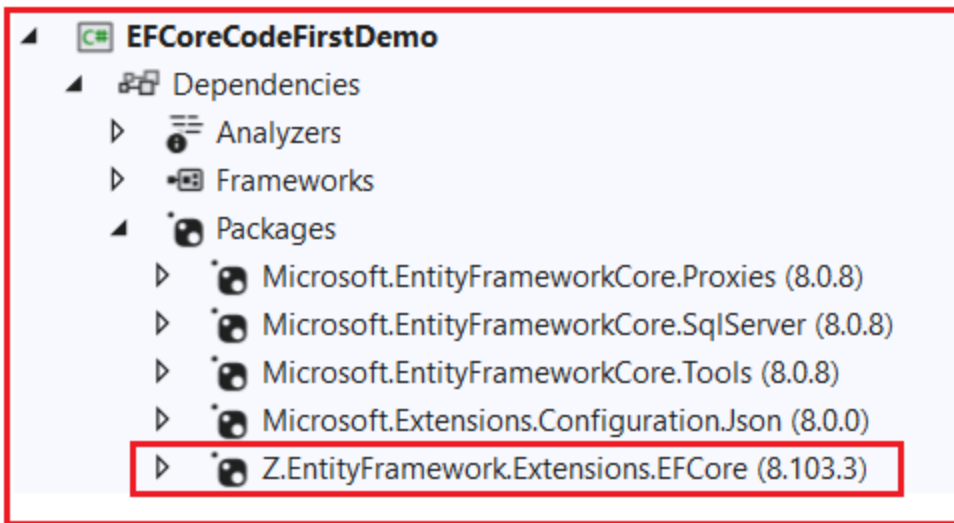
- **BulkInsert / BulkInsertAsync:** Inserts a large number of records into the database in a single operation. This minimizes the overhead of individual inserts.
- **BulkUpdate / BulkUpdateAsync:** Updates multiple records at once by grouping them into a single database transaction.
- **BulkDelete / BulkDeleteAsync:** Deletes multiple records in one batch instead of executing multiple individual delete commands.
- **BulkMerge / BulkMergeAsync:** Combines insert and update operations in one go (similar to a UPSERT)
- **BulkSaveChanges / BulkSaveChangesAsync:** Batches multiple INSERT, UPDATE, and DELETE operations when saving changes, significantly reducing the number of database round trips and improving performance for large-scale modifications.

Install Z.EntityFramework.Extensions.EFCore

First, we need to install the Z.EntityFramework.Extensions.EFCore package in our project. You can install it via NuGet Package Manager or by executing the following command in the Package Manager Console:

[Install-Package Z.EntityFramework.Extensions.EFCore](#)

Once you install the **Z.EntityFramework.Extensions.EFCore** package, you can verify it inside the Packages folder, as shown in the image below.



Z.EntityFramework.Extensions.EFCore Extensions extends our DbContext with high-performance bulk operations: BulkSaveChanges, BulkInsert, BulkUpdate, BulkDelete, BulkMerge, and more. It Supports SQL Server, MySQL, Oracle, PostgreSQL, SQLite, and more. For more information, please check the two links below.

- **GitHub Link:** <https://www.nuget.org/packages/Z.EntityFramework.Extensions.EFCore/>
- **Official Website:** <https://entityframework-extensions.net/bulk-extensions>

Setting Up the Entity and DbContext

We will use a simple Student entity to demonstrate bulk operations using Entity Framework Extensions and configure the DbContext accordingly.

Student Entity

So, create a class file named **Student.cs** within the Entities folder and then copy and paste the following code. StudentId serves as the primary key, and FirstName, LastName, and Branch Properties represent student details.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Branch { get; set; }
    }
}
```

Configuring the DbContext

Modify your EFCoreDbContext class to include the Students DbSet property, which will be mapped to the Students table, and configure the database connection. So, modify the **EFCoreDbContext** class as follows:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging;
namespace EFCoreCodeFirstDemo.Entities
```

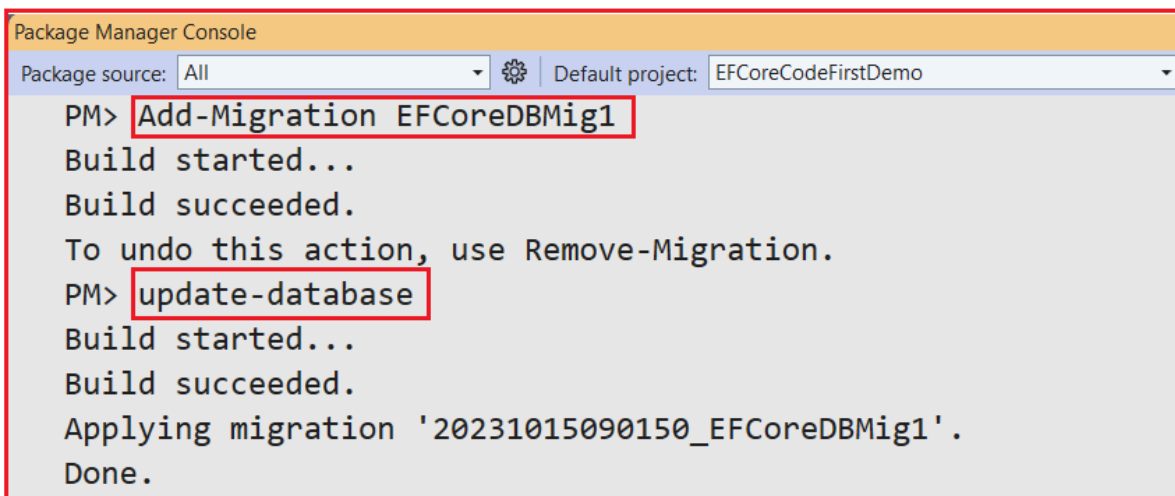
```

{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Optional: Log generated SQL to the console for debugging
            // Uncomment the following line to enable SQL logging
            // optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
            // Configure the SQL Server connection string
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        // DbSet representing the Students table
        public DbSet<Student> Students { get; set; }
    }
}

```

Applying Migrations and Updating the Database

After setting up the Student entity and DbContext, apply migrations to create the database schema. Open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows.

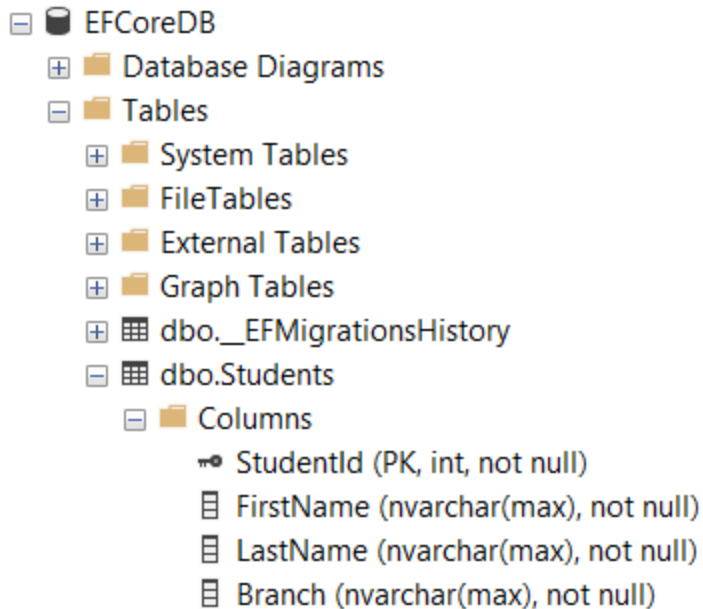


```

Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> Add-Migration EFCoreDBMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20231015090150_EFCoreDBMig1'.
Done.

```

These commands will create the necessary tables in the specified database based on your Student entity, as shown in the below image:



Performing Bulk Insert Operations using Entity Framework Core Extension

The **Z.EntityFramework.Extensions.EFCore** Extensions provides the **BulkInsert** and **BulkInsertAsync** methods to efficiently insert large numbers of entities into the database in a single operation. This approach minimizes database round trips and enhances performance.

For a better understanding, please modify the Program class as follows. In the example below, we insert multiple students into the database using the BulkInsert Extension Method. We don't need to call the **SaveChanges** method while performing the Bulk Insert Operations.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Create a list of new students to insert
                List<Student> newStudents = new List<Student>()
                {
                    new Student() { FirstName = "Pranaya", LastName = "Rout",
Branch = "CSE" },
                    new Student() { FirstName = "Hina", LastName = "Sharma",
Branch = "CSE" },
                    new Student() { FirstName = "Anurag", LastName =
"Mohanty", Branch = "CSE" },
                    new Student() { FirstName = "Prity", LastName = "Tiwary",
Branch = "ETC" }
                };
                using var context = new EFCoreDbContext();
```

```

        // Perform Bulk Insert using EF Extensions
        // Inserts all Student entities in the newStudents list into
the database in a single, optimized operation.
        context.BulkInsert(newStudents);
        //No Need for SaveChanges():
        //The BulkInsert method handles database interactions
internally, eliminating the need to call SaveChanges().
        Console.WriteLine("BulkInsert: Successfully inserted new
students.");

        // Display all students belonging to the CSE branch
        DisplayStudentsByBranch("CSE");
        Console.ReadKey();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"BulkInsert Error: {ex.Message}");
    }
}
// Retrieves and displays students from a specified branch.
public static void DisplayStudentsByBranch(string branch)
{
    using var context = new EFCoreDbContext();
    // Fetch all students where Branch matches the specified value
    var studentsList = context.Students
        .AsNoTracking() // Improves performance
for read-only operations
        .Where(std => std.Branch == branch)
        .ToList();
    Console.WriteLine($"\\nStudents in {branch} Branch:");
    foreach (var student in studentsList)
    {
        Console.WriteLine($"\\tStudent ID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
    }
}
}
}
}

```

Explanation:

- **Creating Students:** A list of Student objects is created to represent the new records to be inserted.
- **BulkInsert:** The BulkInsert method is called on the DbContext to insert all students in a single, optimized operation.
- **Display:** After insertion, the DisplayStudentsByBranch method retrieves and displays students from the specified branch to verify the operation.

Output:

```
BulkInsert: Successfully inserted new students.
```

```
Students in CSE Branch:
```

```
Student ID: 1, Name: Pranaya Rout, Branch: CSE
```

```
Student ID: 2, Name: Hina Sharma, Branch: CSE
```

```
Student ID: 3, Name: Anurag Mohanty, Branch: CSE
```

Performing Bulk Update Operations using EF Core Extension

The `Z.EntityFramework.Extensions.EFCore` provides two methods, `BulkUpdate` and `BulkUpdateAsync`, which allow us to update a large number of entities in the database in a single operation.

For a better understanding, please modify the Program class as follows. In the example below, first, we fetch all the students whose branch is CSE, update the first and last names, and finally update the updated data in the database using the BulkUpdate Extension Method.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkUpdate Operation...");
                // Specify the branch to update
                string branchToUpdate = "CSE";
                // Perform Bulk Update
                BulkUpdateStudents(branchToUpdate);
                Console.WriteLine("BulkUpdate: Successfully updated student
records.");
                // Display updated students to verify changes
                DisplayStudentsByBranch(branchToUpdate);
                Console.ReadKey();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"BulkUpdate Error: {ex.Message}");
            }
        }
        //Updates the first and last names of students in the specified
        branch.
        public static void BulkUpdateStudents(string branch)
        {
            using var context = new EFCoreDbContext();
            // Fetch students belonging to the specified branch
            var studentsToUpdate = context.Students
                .Where(std => std.Branch == branch)
```

```

        .ToList();
    // Modify the desired properties for each student
    foreach (var student in studentsToUpdate)
    {
        student.FirstName += " Updated";
        student.LastName += " Updated";
    }
    // Perform Bulk Update using EF Extensions
    context.BulkUpdate(studentsToUpdate);
}
// Retrieves and displays students from a specified branch.
public static void DisplayStudentsByBranch(string branch)
{
    using var context = new EFCoreDbContext();
    // Fetch all students where Branch matches the specified value
    var studentsList = context.Students
        .AsNoTracking() // Improves performance
        for read-only operations

        .Where(std => std.Branch == branch)
        .ToList();
    Console.WriteLine($"
Updated Students in {branch} Branch:");
    foreach (var student in studentsList)
    {
        Console.WriteLine($"
\tID: {student.StudentId}, First Name:
{student.FirstName}, Last Name: {student.LastName}, Branch:
{student.Branch}");
    }
}
}
}
}

```

Explanation:

- **Fetching Students:** Retrieves all students from the specified branch (CSE in this case).
- **Modifying Records:** Appends " Updated" to both the FirstName and LastName of each student.
- **BulkUpdate:** The BulkUpdate method is called to apply all changes in a single, efficient operation.
- **Display:** The updated records are retrieved and displayed to verify the bulk update.

Output:

```

Starting BulkUpdate Operation...
BulkUpdate: Successfully updated student records.

Updated Students in CSE Branch:
ID: 1, First Name: Pranaya Updated, Last Name: Rout Updated, Branch: CSE
ID: 2, First Name: Hina Updated, Last Name: Sharma Updated, Branch: CSE
ID: 3, First Name: Anurag Updated, Last Name: Mohanty Updated, Branch: CSE

```

Performing Bulk Delete Operation using Entity Framework Core Extension

The **BulkDelete** and **BulkDeleteAsync** methods extend our DbContext object, which allows us to delete a large number of entities from the database with a single round trip, improving the application's performance. For a better understanding, please modify the Program class as follows: In the below example, first, we fetch all the students where the Branch is CSE, and then we delete the retrieved student using the BulkDelete Extension Method.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
using System;
using System.Linq;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkDelete Operation...");
                // Specify the branch to delete students from
                string branchToDelete = "ETC";
                // Perform Bulk Delete
                BulkDeleteStudents(branchToDelete);
                Console.WriteLine("BulkDelete: Successfully deleted student
records.");

                // Display remaining students to verify deletion
                DisplayStudentsByBranch("CSE");
                DisplayStudentsByBranch("ETC");
                Console.ReadKey();
            }
            catch (Exception ex)
            {
                Console.WriteLine($"BulkDelete Error: {ex.Message}");
            }
        }
        // Deletes all students belonging to the specified branch.
        public static void BulkDeleteStudents(string branch)
        {
            using var context = new EFCoreDbContext();
            // Fetch students belonging to the specified branch
            var studentsToDelete = context.Students
                .Where(std => std.Branch == branch)
                .ToList();
            // Perform Bulk Delete using EF Extensions
            context.BulkDelete(studentsToDelete);
        }
        // Retrieves and displays students from a specified branch.
        public static void DisplayStudentsByBranch(string branch)
        {
            using var context = new EFCoreDbContext();
            // Fetch all students where Branch matches the specified value
            var studentsList = context.Students
```

```

        .AsNoTracking() // Improves performance
for read-only operations

        .Where(std => std.Branch == branch)
        .ToList();

    if (studentsList.Any())
    {
        Console.WriteLine($"\\nStudents in {branch} Branch:");
        foreach (var student in studentsList)
        {
            Console.WriteLine($"\\tID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
        }
    }
    else
    {
        Console.WriteLine($"\\nNo students found in {branch}
Branch.");
    }
}
}
}
}

```

Explanation:

- **Fetching Students:** Retrieves all students from the specified branch (ETC in this case).
- **BulkDelete:** The BulkDelete method removes all fetched records in a single operation.
- **Display:** Attempts to display students from both CSE and ETC branches to verify the deletion.

Output:

```

Starting BulkDelete Operation...
BulkDelete: Successfully deleted student records.

Students in CSE Branch:
    ID: 1, Name: Pranaya Updated Rout Updated, Branch: CSE
    ID: 2, Name: Hina Updated Sharma Updated, Branch: CSE
    ID: 3, Name: Anurag Updated Mohanty Updated, Branch: CSE

No students found in ETC Branch.

```

Asynchronous Bulk Operations with EF Core Extensions

Asynchronous operations enhance application responsiveness by ensuring database interactions do not block the main thread. Z.EntityFramework.Extensions.EFCore supports asynchronous bulk operations, allowing for non-blocking data manipulation. Before proceeding further, please delete all the data from the Students table by executing the following TRUNCATE statement.

TRUNCATE TABLE Students;

Next, modify the Program class as follows to perform the Bulk Operations asynchronously using Entity Framework Core Extension. In the example below, we use asynchronous methods like BulkInsertAsync, BulkUpdateAsync, and BulkDeleteAsync to ensure the main thread remains responsive and unblocked during execution. The following example code is self-explained, so please read the comment lines for a better understanding.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            Console.WriteLine("=== EF Core Asynchronous CRUD Operations with
EF Extensions ===\n");
            try
            {
                // Initialize the DbContext
                using var context = new EFCoreDbContext();
                // 1. Create (Bulk Insert) Operation
                await BulkInsertStudentsAsync(context);
                // 2. Read Operation
                await DisplayStudentsByBranchAsync(context, "CSE");
                // 3. Update (Bulk Update) Operation
                await BulkUpdateStudentsAsync(context, "CSE");
                // 4. Read Operation to verify updates
                await DisplayStudentsByBranchAsync(context, "CSE");
                // 5. Delete (Bulk Delete) Operation
                await BulkDeleteStudentsAsync(context, "ETC");
                // 6. Read Operations to verify deletions
                await DisplayStudentsByBranchAsync(context, "CSE");
                await DisplayStudentsByBranchAsync(context, "ETC");
            }
            catch (Exception ex)
            {
                // Handle any unexpected exceptions
                Console.WriteLine($" \nAn unexpected error occurred:
{ex.Message}");
            }
            // Performs bulk insert of new students asynchronously.
            private static async Task BulkInsertStudentsAsync(EFCoreDbContext
context)
            {
                Console.WriteLine("1. Starting Bulk Insert Operation...");
                // Define a list of new students to insert
                List<Student> newStudents = new List<Student>()
                {
                    new Student() { FirstName = "Alice", LastName = "Johnson",
Branch = "CSE" },

```

```

        new Student() { FirstName = "Bob", LastName = "Smith", Branch
= "CSE" },
        new Student() { FirstName = "Charlie", LastName = "Brown",
Branch = "ETC" },
        new Student() { FirstName = "Diana", LastName = "Prince",
Branch = "ETC" }
    };
    try
    {
        // Perform Bulk Insert asynchronously
        await context.BulkInsertAsync(newStudents);
        Console.WriteLine("Bulk Insert: Successfully inserted new
students.\n");
    }
    catch (Exception ex)
    {
        // Handle exceptions related to bulk insert
        Console.WriteLine($"Bulk Insert Error: {ex.Message}\n");
    }
}
// Displays students belonging to a specific branch asynchronously.
private static async Task
DisplayStudentsByBranchAsync(EFCoreDbContext context, string branch)
{
    Console.WriteLine($"2. Retrieving Students in '{branch}'
Branch...\n");
    try
    {
        // Fetch students asynchronously with no tracking for
performance
        var studentsList = await context.Students
                                .AsNoTracking()
                                .Where(std => std.Branch ==
branch)
                                .ToListAsync();

        if (studentsList.Any())
        {
            Console.WriteLine($"Students in '{branch}' Branch:");
            Console.WriteLine("-----");
            foreach (var student in studentsList)
            {
                Console.WriteLine($"  ID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
            }
            Console.WriteLine();
        }
        else
        {
            Console.WriteLine($"No students found in '{branch}'
Branch.\n");

```

```

    }
}
catch (Exception ex)
{
    // Handle exceptions related to data retrieval
    Console.WriteLine($"Read Error: {ex.Message}\n");
}
}
// Performs bulk update of students' names asynchronously.
private static async Task BulkUpdateStudentsAsync(EFCoreDbContext
context, string branch)
{
    Console.WriteLine("3. Starting Bulk Update Operation...");
    try
    {
        // Fetch students asynchronously
        var studentsToUpdate = await context.Students
                                                .Where(std => std.Branch
== branch)
                                                .ToListAsync();

        if (studentsToUpdate.Any())
        {
            // Modify the desired properties for each student
            foreach (var student in studentsToUpdate)
            {
                student.FirstName += " Updated";
                student.LastName += " Updated";
            }
            // Perform Bulk Update asynchronously
            await context.BulkUpdateAsync(studentsToUpdate);
            Console.WriteLine("Bulk Update: Successfully updated
student records.\n");
        }
        else
        {
            Console.WriteLine($"No students found in '{branch}'
Branch to update.\n");
        }
    }
    catch (Exception ex)
    {
        // Handle exceptions related to bulk update
        Console.WriteLine($"Bulk Update Error: {ex.Message}\n");
    }
}
// Performs bulk delete of students belonging to a specific branch
asynchronously.
private static async Task BulkDeleteStudentsAsync(EFCoreDbContext
context, string branch)
{
    Console.WriteLine("4. Starting Bulk Delete Operation...");

```

```

        try
        {
            // Fetch students asynchronously
            var studentsToDelete = await context.Students
                                                .Where(std => std.Branch
== branch)
                                                .ToListAsync();

            if (studentsToDelete.Any())
            {
                // Perform Bulk Delete asynchronously
                await context.BulkDeleteAsync(studentsToDelete);
                Console.WriteLine($"Bulk Delete: Successfully deleted
students from '{branch}' Branch.\n");
            }
            else
            {
                Console.WriteLine($"No students found in '{branch}'
Branch to delete.\n");
            }
        }
        catch (Exception ex)
        {
            // Handle exceptions related to bulk delete
            Console.WriteLine($"Bulk Delete Error: {ex.Message}\n");
        }
    }
}

```

Explanation of the Code:

- **Bulk Insert:** Inserts four students into the database.
- **Read Operation:** Displays students in the CSE branch.
- **Bulk Update:** Appends " Updated" to the first and last names of students in the CSE branch.
- **Read Operation:** Verifies that the update was successful.
- **Bulk Delete:** Deletes students in the ETC branch.
- **Read Operations:** Confirms the deletion by attempting to retrieve students from both CSE and ETC branches.

Output:

```
=== EF Core Asynchronous CRUD Operations with EF Extensions ===
```

```
1. Starting Bulk Insert Operation...
```

```
Bulk Insert: Successfully inserted new students.
```

```
2. Retrieving Students in 'CSE' Branch...
```

```
Students in 'CSE' Branch:
```

```
-----
```

```
    ID: 1, Name: Alice Johnson, Branch: CSE
```

```
    ID: 2, Name: Bob Smith, Branch: CSE
```

```
3. Starting Bulk Update Operation...
```

```
Bulk Update: Successfully updated student records.
```

```
2. Retrieving Students in 'CSE' Branch...
```

```
Students in 'CSE' Branch:
```

```
-----
```

```
    ID: 1, Name: Alice Updated Johnson Updated, Branch: CSE
```

```
    ID: 2, Name: Bob Updated Smith Updated, Branch: CSE
```

```
4. Starting Bulk Delete Operation...
```

```
Bulk Delete: Successfully deleted students from 'ETC' Branch.
```

```
2. Retrieving Students in 'CSE' Branch...
```

```
Students in 'CSE' Branch:
```

```
-----
```

```
    ID: 1, Name: Alice Updated Johnson Updated, Branch: CSE
```

```
    ID: 2, Name: Bob Updated Smith Updated, Branch: CSE
```

```
2. Retrieving Students in 'ETC' Branch...
```

```
No students found in 'ETC' Branch.
```

What is BulkMerge?

BulkMerge combines the functionalities of BulkInsert and BulkUpdate into a single operation, often called an “UPSERT.” It can insert new records and update existing ones based on specified conditions or primary keys. BulkMerge internally determines whether each entity in the collection should be inserted or updated.

When to Use BulkMerge:

- **Mixed Data Sets:** When you have a combination of new and existing records to process.
- **Simplified Operations:** When you want to handle insertions and updates in a single, efficient operation without managing them separately.
- **Performance Optimization:** Reduces the number of database calls by handling multiple operations in one batch.

BulkMerge Example using Student Entity

Please modify the Program class as follows. In this example, we will bulk merge a list of students with existing data in the database. The method will update the existing students if they match by StudentId and insert new ones.

```

using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkMerge operation...");
                // Perform Bulk Merge operation
                await BulkMergeAsync();
                Console.WriteLine("BulkMerge operation completed.");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"An error occurred: {ex.Message}");
            }
        }
        private static async Task BulkMergeAsync()
        {
            using var context = new EFCoreDbContext();
            // Existing list of students to merge (sync) with the database
            List<Student> studentsToMerge = new List<Student>
            {
                // Assume that StudentId 1 exists and will be updated
                new Student { StudentId = 1, FirstName = "John", LastName =
"Doe Updated", Branch = "CSE" },

                // This is a new student that will be inserted
                new Student { FirstName = "Ramesh", LastName = "Sethy",
Branch = "CSE" }
            };
            // Perform BulkMerge operation:
            // - Updates student with StudentId = 1
            // - Inserts new student without an ID
            await context.BulkMergeAsync(studentsToMerge);
            // Display current students in the database after the merge
            await DisplayStudentsAsync();
        }
        private static async Task DisplayStudentsAsync()
        {
            using var context = new EFCoreDbContext();
            var students = await context.Students.ToListAsync();
            Console.WriteLine("Current Students in the database:");
            foreach (var student in students)
            {
                Console.WriteLine($"  \tID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
            }
        }
    }
}

```

```

        Console.WriteLine(); // Blank line for better readability
    }
}
}

```

Explanation:

Preparing Data:

- A Student object with StudentId = 1 is provided. Assuming this ID exists in the database, this record will be updated.
- A Student object without a StudentId is provided. This record will be inserted as a new entry.

BulkMergeAsync:

The BulkMergeAsync method processes the list:

- Updates the existing student with StudentId = 1.
- Adds the new student to the database.

Display:

- After the merge, all students in the database are retrieved and displayed to verify the operation.

Output:

```

Starting BulkMerge operation...
Current Students in the database:
    ID: 1, Name: John Doe Updated, Branch: CSE
    ID: 2, Name: Bob Updated Smith Updated, Branch: CSE
    ID: 5, Name: Ramesh Sethy, Branch: CSE

BulkMerge operation completed.

```

What is BulkSaveChanges?

BulkSaveChanges is an extension method that batches multiple insert, update, and delete operations into a single round trip to the database. This method is particularly useful when we have numerous changes to entities within the context, and we want to save them all efficiently at once, thereby enhancing the performance of SaveChanges for large datasets.

When to Use BulkSaveChanges:

- **Multiple Operations:** When our context object contains a mix of insertions, updates, and deletions entities.
- **Performance Optimization:** Batching operations reduce the number of database calls, which is especially beneficial for large-scale data modifications.
- **Transactional Consistency:** Ensures that all batched operations are executed within a single transaction, maintaining data integrity.

BulkSaveChanges Example using Student Entity

In this example, we will demonstrate how to update, insert, and delete multiple students and then use BulkSaveChanges to save all changes efficiently in one go.

```
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static async Task Main(string[] args)
        {
            try
            {
                Console.WriteLine("Starting BulkSaveChanges operation...");
                // Perform Bulk SaveChanges operation
                await BulkSaveChangesAsync();
                Console.WriteLine("BulkSaveChanges operation completed.");
            }
            catch (Exception ex)
            {
                Console.WriteLine($"An error occurred: {ex.Message}");
            }
        }
        private static async Task BulkSaveChangesAsync()
        {
            using var context = new EFCoreDbContext();
            // Fetch existing students from the database
            var existingStudents = await context.Students.Where(s => s.Branch
== "CSE").ToListAsync();
            // Updating existing students (append 'Updated' to their names)
            foreach (var student in existingStudents)
            {
                student.FirstName += " Updated";
                student.LastName += " Updated";
            }
            // Adding a new student to the context
            context.Students.Add(new Student { FirstName = "New", LastName =
"Student", Branch = "ETC" });
            // Deleting a student from the context (this will be batched in
the BulkSaveChanges)
            var studentToDelete = await context.Students.Where(s =>
s.FirstName == "Sethy").FirstOrDefaultAsync();
            if (studentToDelete != null)
            {
                context.Students.Remove(studentToDelete);
            }
            // Perform BulkSaveChanges to save all updates, inserts, and
deletes in one go
```

```

        await context.BulkSaveChangesAsync();
        // Display current students in the database after the save
        await DisplayStudentsAsync();
    }
    private static async Task DisplayStudentsAsync()
    {
        using var context = new EFCoreDbContext();
        var students = await context.Students.ToListAsync();
        Console.WriteLine("Current Students in the database:");
        foreach (var student in students)
        {
            Console.WriteLine($"ID: {student.StudentId}, Name:
{student.FirstName} {student.LastName}, Branch: {student.Branch}");
        }
        Console.WriteLine(); // Blank line for better readability
    }
}
}

```

When you run the application, you will observe that the student with the First Name “Sethy” has been deleted, and the new student “New Student” has been added. Existing students in the CSE branch have their names updated accordingly.

Licensing:

- **Community Edition (Free):** Offers basic bulk operation functionalities suitable for many applications.
- **Professional and Enterprise Editions (Paid):** Provide advanced features, performance enhancements, and dedicated support. Licensing fees apply based on the edition and usage requirements.

Z.EntityFramework.Extensions.EFCore enhances EF Core by providing efficient bulk operations for applications that handle large amounts of data. These methods can optimize database interactions and improve application performance.

In the next article, I will discuss [Bulk Operations in Entity Framework Core using EFCore.BulkExtensions](#) with Examples. In this article, I explain Bulk Operations in Entity Framework Core using Z.EntityFramework.Extensions.EFCore with examples. I hope you enjoyed this Bulk Operations in Entity Framework Core using Z.EntityFramework.Extensions.EFCore article.